Regression-Gated Skill Learning: Label-Free Promotion Gates for Self-Improving LLM Agents

Hwuiwon Kim
Independent Researcher
hwuiwon.kim@gmail.com

Abstract

We present *regression-gated skill learning*, a mechanism by which an LLM agent platform distills reusable skills from its own successful task executions and promotes them only after they pass automatically generated regression tests. Letting agents learn from their own behavior in production is risky precisely where it is most useful: promotion decisions have no ground-truth labels, LLM judges are nondeterministic and gameable, and a skill distilled from a session trivially passes any test generated from that same session. Contemporaneous systems increasingly gate skill or harness updates on held-out evaluation, but presuppose an evaluation signal: benchmark outcomes, verifier scores, or judge preferences. Our contribution is one layer down: *constructing the test oracle itself* from ordinary unlabeled traffic. A deterministic *grounded-facts oracle* extracts typed spans (numbers, identifiers, URLs, paths, quoted text) from the final response, admits only those corroborated word-boundary-exactly in the session's successful tool outputs, and persists them as regression assertions; *sibling suites* accumulated from other sessions of the same recurring task supply held-out material the candidate never saw; and a paired non-regression comparison against the serving revision feeds human review, append-only provenance, and a post-promotion rollback monitor. The mechanism is implemented in MOA, a multi-tenant enterprise agent platform, and we characterize it with a deterministic case study that drives the implementation's real code paths on 1,400 synthetic task segments. Spec-conformance checks confirm 100% detection of string-level fact fabrication and omission, and an adversarial study maps the matcher's precision boundary: zero false groundings under alphanumeric extensions, but systematic false groundings under punctuation extensions (`$1,200` grounding against `$1,200.99`), motivating typed canonicalization. The empirical finding is selectivity: the oracle accepts 92.5% of planted-fact-preserving paraphrases that a keyword-matching fallback rejects entirely, while tolerating alternative tool orderings over the same tool set.

1 Introduction

An LLM agent that performs the same task repeatedly should get better at it. The obvious mechanism is well established in research settings: distill successful executions into reusable procedural documents ("skills") and inject them into future context. Voyager grows a library of verified game skills [38], Agent Workflow Memory induces workflows from web trajectories [41], and ExpeL distills insights from experience pools [48]. In production, however, this loop acquires failure modes that benchmark environments do not surface. There is no reward function: nobody labels which of ten thousand support sessions were actually resolved. There is no free re-execution: replaying a candidate skill against live enterprise systems to see whether it works is exactly the kind of side effect

Preprint.

the platform exists to prevent. And there is a subtle overfitting trap: a skill distilled *from* a session will, almost by construction, pass any test generated from that same session—a held-out split that holds nothing out.

The known alternatives each fail one of these constraints. LLM-as-judge scoring is nondeterministic, biased, and gameable by the very models it evaluates [50, 39]; using it to gate self-modification makes the judge a proxy objective, inviting the gaming behavior documented for misspecified proxies in RL [25]. Execution-based verification [38, 9] needs a sandbox that faithfully reproduces the production environment, which for an agent whose tools touch tenant databases and third-party APIs does not exist. Benchmark-score gating, as in the Darwin Gödel Machine [47], imports the benchmark's blind spots into the improvement loop; repeated evaluation against a fixed test set risks adaptive overfitting [5], and even absent adaptivity, a single static test set can mislead about generalization [28].

This paper describes how MOA, a multi-tenant enterprise agent platform, closes the loop anyway. The central idea is that a successful session already contains its own verification signal: the facts the agent reported to the user (latencies, ticket numbers, file paths, order totals) either do or do not appear in the outputs of the tools it ran. A response that carries tool-corroborated facts forward has carried real tool-derived evidence to the user; a response that cannot be corroborated has not. MOA operationalizes this as a *grounded-facts oracle*: at proposal time, it deterministically extracts typed spans from the final assistant response, keeps only those that occur word-boundary-exactly in the session's successful tool outputs, and emits them as blocking assertions in a generated regression suite. The oracle is an *evidence-faithfulness* signal, not a semantic-correctness verifier (§4.2 states the exact guarantee). What it buys is independence: no human task labels, no reference answers, no external reward, no LLM judge, and a construction that is deterministic and auditable.

Around this oracle, MOA builds a promotion lifecycle borrowed from software release engineering rather than from reinforcement learning [7, 31]. Candidate skills are filed as reviewable proposals with typed provenance. The review-time gate runs the candidate's generated suite *and* a held-out pool assembled from *sibling suites* (suites generated from other sessions of the same recurring task, which the candidate's draft never saw) under a strict non-regression comparator. Promotion is a human-approved, transactional move of a serving pointer; every learning event lands in an append-only, bitemporal log; and a deterministic post-promotion monitor compares resolution rates before and after activation and files rollback proposals when a promoted skill underperforms. A sanitization gate makes raw transcripts unrepresentable in the learning path: nothing that has not passed PII classification can reach distillation, suite generation, or storage.

Contributions.

- We describe a complete, implemented architecture for *regression-gated skill learning*: experience assessment, sanitization, LLM distillation, deterministic suite generation, a two-sided (held-in/held-out) promotion gate, human review, append-only provenance, and rollback monitoring (§3–§4).
- We define the *grounded-facts oracle*, a deterministic mechanism for constructing regression test oracles from unlabeled traces: typed response spans become blocking assertions only when corroborated by successful tool-result evidence. It requires no human task labels, reference answers, external reward, or LLM judge; its guarantee is *evidence faithfulness* (preservation of tool-supported literals), not semantic correctness (§4.2).
- We show how *held-out sibling suites* structurally prevent direct same-session test leakage (the candidate's own suite is excluded from the held-out pool by storage-level construction) and identify the leakage modes that remain (§4.3).
- We characterize the oracle with a deterministic case study through the production code paths (1,400 synthetic segments, 2,000 gate evaluations): 100% detection of string-level fact fabrication and omission; an adversarial precision map showing zero false groundings under alphanumeric extensions but systematic false groundings under punctuation extensions, motivating typed canonicalization; 92.5% acceptance of planted-fact-preserving paraphrases versus 0% for a keyword baseline; and tolerance of alternative tool orderings over the same tool set (§5). We also report design lessons from building the system, including why a trajectory-similarity gate was deleted (§6).

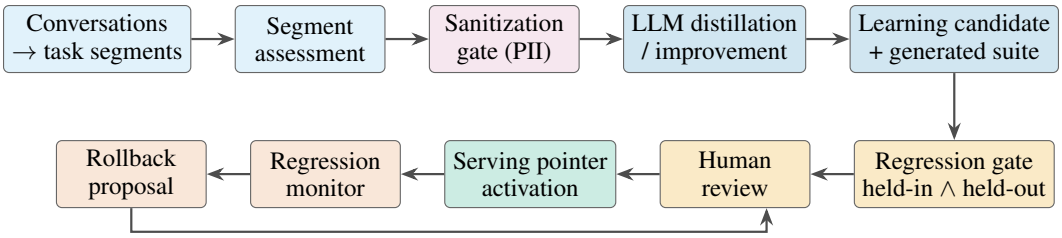

Figure 1: The regression-gated skill learning loop. Successful task segments are assessed, sanitized, and distilled into candidate skills; each candidate must clear a paired non-regression gate over its own generated suite *and* a held-out pool of sibling suites before human review moves the serving pointer. A monitor watches post-promotion resolution rates and files rollback proposals.

2 Related work

Skill libraries and experience learning. A growing line of work has agents accumulate reusable procedural knowledge from their own trajectories: executable skill libraries verified by environment execution [38], workflow induction from web navigation [41], distilled cross-task insights [48], interactively learned environment manuals [11], self-synthesized and practiced API skills [49], computer-control skills from screen interaction [35], and continually updated causal memories [19]. MOA shares the distill-experience-into-artifacts premise (in the SKILL.md package format [4, 1]) but differs on the admission side: in these systems validation is execution success in a sandboxed environment, LLM self-judgment, or downstream benchmark reward, and none interposes a governed release lifecycle (human review, typed provenance, monitored rollback, and a sanitization boundary that keeps raw transcripts out of the learning path) between distillation and serving.

Contemporaneous regression-gated skill evolution. In the months preceding this preprint, the admission side became an active direction of its own. GRASP treats skills as gated edits admitted only after a balanced held-out probe under a regression budget [22]; SkillOpt accepts a skill edit only when it strictly improves held-out validation performance [43]; Self-Harness admits harness modifications via held-in plus held-out regression testing with a conservative no-degradation rule and repeated evaluation under stochasticity [46]; and the Regression Tax quantifies empirically that skills materially regress previously solved tasks [36], which is the failure mode our comparator exists to catch. Ground-truth-free admission also has direct precedents: SkillAudit evolves skills without external ground truth via paired trajectory auditing against a fixed structural verifier [12], and Self-Supervised Skill Optimization gates updates on an unlabeled validation set using comparative LLM judgments [26]. Held-out gating of self-modification is therefore no longer novel in itself. Our focus is complementary and one layer down: these systems presuppose an evaluation signal (benchmark outcomes, verifier scores, or judge preferences), whereas we construct the *test oracle itself* from ordinary unlabeled traffic, by deterministically corroborating typed response spans against successful tool-result evidence and persisting them as regression assertions inside a governed release lifecycle. ProvenanceGuard's observation that pooled evidence can support a claim while misattributing which source supports it [2] sharpens a limitation our concatenated grounding corpus shares (§8).

Self-improvement and its safety. Self-refinement and self-training methods [18, 33, 45] and, at the extreme, self-modifying agent populations gated by benchmark scores [47] form a fast-moving space with dedicated surveys [37, 13]. The accompanying safety literature documents model collapse from self-generated data [34] and reward hacking of proxy objectives [25]. MOA takes these as design constraints: improvement is confined to auditable declarative documents (never weights, never raw trajectories), the gate signal is exact corroboration rather than a learned or judged score, generalization is measured on held-out sibling sessions, and every promotion is reversible.

Verifying agent outputs without labels. Existing label-free verification uses LLM judges, whose biases are well documented [50, 39]; agreement- and execution-based proxies [40, 9]; or evidence-grounding checks developed for attribution: the AIS framework [27], attributed QA [6], citation evaluation [15], post-hoc revision [14], sampling-based hallucination detection [20], and atomic-fact factual-precision scoring [21]. The grounded-facts oracle belongs to the evidence-grounding family

but repurposes it: rather than scoring attribution quality of free text against retrieved documents with NLI models or judges, it extracts high-precision typed spans and requires exact word-boundary corroboration in the session's own tool outputs, yielding a deterministic pass/fail oracle used not to score single responses but to *generate regression suites* that gate promotion.

Behavioral testing and safe deployment. CheckList brought suite-based behavioral testing to NLP models [29], and the ML Test Score codified testing and monitoring for production pipelines [7]; a parallel thread studies regressions from model updates: negative flips [42], update regression [8], and behavioral drift of deployed LLM APIs [10]. Deriving test oracles from observed executions has a long software-engineering lineage [23], and benchmarks for tool-agent interaction increasingly stress verifiable end states [44]. The MLOps literature motivates staged rollout, monitoring, and rollback [31, 24, 32, 3]. MOA transplants this discipline to in-context skill artifacts: generated suites are pre-deployment tests derived from real traffic rather than hand-written templates, sibling suites act as shadow evaluation, the resolution-rate monitor is the canary metric, and rollback proposals close the loop. Concerns about benchmark overfitting and contamination [30, 17, 16] appear here as an internal threat, partially addressed architecturally (§4.3).

3 System overview

MOA is a multi-tenant enterprise agent platform written in Rust, with durable orchestration on Restate and tenant state in Postgres. Four properties of the surrounding platform matter for this paper. First, every conversation is an append-only event log; tool calls, tool results, approvals, and responses are typed events that can be replayed. Second, work is measured in *task segments*: contiguous spans of a session bounded by task-boundary signals, each closed by a deterministic *segment assessment* that scores resolution from five weighted signal classes (tool outcomes, verification events, user continuation behavior, model self-assessment, structural features) into an outcome in $\{$`Resolved`, `Partial`, `Unknown`, `Failed`, `Abandoned`$\}$ with a confidence score. Third, all learning state is tenant-scoped: skills, candidates, and logs never cross tenant boundaries. Fourth, every learned behavior is required to have provenance and a rollback path, a platform design value that predates and motivates the mechanism described here.

Resolved segments become *experience records*: task fingerprint, facets, tools used, outcome, confidence, and typed attributions of which skills, tools, or memories helped or harmed. Experiences are the sole input to skill learning. The complete loop (Figure 1) is: experiences are sanitized and distilled into candidate skills; candidates are filed with a deterministically generated regression suite; the review-time gate executes that suite and a held-out sibling pool; a human reviewer accepts or rejects; acceptance transactionally activates a new skill revision behind a serving pointer; a monitor watches post-promotion resolution rates and files rollback proposals. Skills are Agent-Skills-style packages (YAML frontmatter plus a Markdown procedure body, optional scripts and references, and the generated suite at `tests/regression-suite.toml`), injected into future context by relevance ranking with progressive disclosure of body and resources.

4 Regression-gated skill learning

4.1 Admission: which experiences may teach

Not every success is worth learning from, and not every learnable success is releasable. A *learnability gate* requires `Resolved` outcomes to carry confidence ≥ 0.7; `Partial` outcomes need confidence ≥ 0.85 *and* a helpful attribution *and* corroborating verification evidence; failed or abandoned segments never teach. A tool-call floor (default 8; relaxed to 3 for recurring tasks) filters out trivial segments whose "procedure" is a single lookup. One assumption should be explicit: `Resolved` is not ground truth. It is the output of the deterministic composite assessment (§3), whose signals include model self-assessment. When we say the pipeline needs no labels, the precise claim is that *no human task labels, reference answers, or external reward are required to construct the promotion assertions*; the admission layer produces heuristic outcome labels, and this paper conditions on that layer identifying learnable experience correctly rather than validating it.

Before any model call, a *sanitization gate* converts the segment's events into a sanitized evidence value, `SanitizedLearningEvidence`. Every carrier passes a PII classifier: user messages, assistant responses and reasoning summaries, tool arguments (walked as JSON, key and value position), tool

results and errors, memory paths, task summaries. Redactions are re-classified for residual sensitivity, and any refusal (restricted class, credential material, classifier abstention) ends the learning pass with zero writes and zero provider calls. The type has no raw-string constructor and no deserializer, so code paths downstream of the gate cannot bypass the sanitizer API to represent unsanitized text. That is a guarantee about the type boundary, not about the classifier: a false negative still passes whatever the classifier missed. This is the structural answer to a compliance question every self-learning production system faces (what if the model memorializes a customer's data into a reusable artifact?), and it also constrains the oracle design in §4.2: expectations must survive redaction.

Distillation is a single auxiliary-model call that turns sanitized evidence into a complete SKILL.md (when-to-use guidance, numbered procedure, pitfalls, verification steps), explicitly instructed to learn durable workflow structure and not to copy transient identifiers. Routing between *creating* a new skill and *improving* an existing one is decided by embedding similarity against serving skills (cosine threshold 0.80, with a lexical Jaccard fallback); near-duplicate proposals are deduplicated onto open candidates, whose suites accumulate as siblings (§4.3). A deterministic *recurrence miner* clusters exact and near-duplicate task fingerprints (embedding union-find, similarity 0.85) and, when a loop recurs ≥ 3 times in 30 days, distills the best exemplar while feeding the remaining members through the same sanitization gate as held-out material. A complementary *weakness miner* clusters recurring failure signals into non-reviewable authoring candidates; mining observes that something keeps failing, but does not produce a change anything can auto-apply.

4.2 The grounded-facts oracle

At proposal time—deterministically, with no model call—MOA generates a regression suite from the source segment. The suite's test case replays the segment's opening user message as input; its blocking assertions are (i) *response-carries-source-facts*: the candidate's response must contain the expected output set, and (ii) *recorded-tools-were-used*: the candidate must use the distinct set of tools the successful run used. A third assertion records the observed tool *order* but is diagnostic-only (§6).

The expected output set is where the verification signal lives. The generator scans the final assistant response, in order of first appearance, for five classes of typed spans: quoted or backticked text (3–80 bytes); URLs; file paths; numbers with optional grouping, currency prefixes, and unit suffixes (`45ms`, `$1,200`, `12%`), excluding trivial bare integers 0–2 and bare years; and identifiers (UUIDs, reference tokens like `ABC-123`, commit hashes, `snake_case`/`camelCase` names). Each candidate fact is tested against a *grounding corpus*: the concatenated text of the session's *successful, non-error* tool results only. Denied calls, failed executions, and tool errors are excluded, so failure identifiers can never become required expectations. A fact grounds if it occurs in the corpus case-insensitively with word boundaries on both sides (a word character being ASCII alphanumeric or underscore): `42` does not ground against `1420`, and `45ms` does not ground against `845ms`, but punctuation-edged facts like `#482` and `$1,200` match exactly. The boundary check protects against alphanumeric extensions only; because `.`, `-`, and `/` are non-word characters, a fact can still ground against a punctuation-extended variant (`$1,200` against `$1,200.99`), a precision gap we measure in §5 and whose fix (typed per-class canonicalization) we outline there. The first five grounded facts (deduplicated, $\leq$80 bytes each) become the expected output; the suite records its oracle kind as `GroundedFacts`. If nothing grounds (no tool results, or a response whose spans none trace to a tool), the generator falls back to a `Keywords` oracle over a fixed subset of the response's longer tokens (length ≥ 5, lowercased, deduplicated) and records that, so reviewers and reports can distinguish fact-grounded cases from fallback cases.

What does a pass certify? `GroundedFacts` verifies *preservation of selected tool-supported literals*: a passing candidate demonstrably carried spans forward that some successful tool result also produced. It is a necessary evidence-faithfulness signal, not a sufficient verifier of semantic correctness or task completion. It does not establish that the tool result was itself correct, that the fact was relevant to the question, that the response interpreted it correctly, that no contradictory tool result existed, or that the task was completed. Corroboration across carriers (response *and* tool output, produced by different mechanisms) is what makes even this weaker signal hard to satisfy vacuously; the surrounding lifecycle (tool-set assertion, held-out siblings, human review, rollback monitor) is what the design relies on for the rest. The oracle is deterministic and auditable: a reviewer reading the suite TOML sees exactly which facts the gate will demand and where they came from.

4.3 The two-sided gate: held-in and held-out

A suite generated from session s cannot, alone, certify a skill distilled from session s; that would grade a draft on the cases it was derived from and report a passing held-out split that held nothing out. MOA therefore accumulates *sibling suites*: when another session of the same recurring task deduplicates onto an open candidate, or a recurrence cluster distills its exemplar, suites generated from those *other* sessions (each independently sanitized) attach to the candidate as held-out contributions, up to a cap of three. Storage enforces the split: the candidate's own suite is recorded as `Generated`; only `Accumulated` sibling rows feed the held-out pool. For an improvement to an existing skill, the pool additionally contains the previous promoted revision's own suite, so the new revision is tested on the material that admitted its predecessor. Two scoping notes. First, this construction *structurally prevents direct same-session test leakage*; it does not prevent all adaptive overfitting. Sibling sessions may be highly similar, predecessor suites are reused across revisions, and a proposer that inspects a held-out failure before revising has partially exposed that test. Second, sibling validity rests on the recurrence-identity mechanism (embedding clustering at fixed thresholds, §4.1): a false merge attaches an unrelated session's suite as bogus held-out material, and a false split forfeits held-out coverage. We do not evaluate clustering accuracy in this paper; the study conditions on it.

At review time, the gate compiles both packages—previous serving revision and candidate—into eval agents that differ *only* in the injected skill markdown (otherwise the score comparison is vacuous), and runs the suites under a fixed evaluator set with a hard cost ceiling (default $0.50 per gate). A candidate whose estimated evaluation cost exceeds the budget is currently rejected outright; this makes gate cost a bounded design parameter, but conflates "candidate is bad" with "we declined to spend enough to evaluate it," and a distinct deferred-evaluation state would be the cleaner design.

Where do tool results come from at review time? Eval agents run against a live model provider with the platform's real tool router over an *isolated temporary sandbox workspace*: never live tenant systems, and no replay of recorded tool outputs. Facts the sandbox can reproduce (file contents, computed values, tool-mediated state the skill itself establishes) discriminate between revisions; environment-dependent facts that cannot reproduce fail both runs equally, and the comparator below neutralizes them. We call the candidate's own suite the *held-in* side of the gate and the sibling/predecessor pool the *held-out* side. Scoring averages blocking assertions only.

The acceptance criterion is not "the candidate passes its tests" but a *paired non-regression comparison against the serving revision*: fewer failed runs wins outright; equal failures require average score $\geq$ baseline. The epsilon in that comparison guards floating-point representation, not model sampling variance; the eval-agent runs themselves are stochastic, and repeated paired trials, as Self-Harness runs under its no-degradation rule [46], would be the statistically principled upgrade. A first revision with no predecessor instead faces an *absolute* criterion (candidate-only execution with zero failures), which exposes an asymmetry: for improvements, an environment-dependent fact that cannot reproduce in the sandbox fails both runs and neutralizes, but a first revision has no paired run to neutralize against, so a valid skill distilled from an environment-dependent result may be structurally unable to clear its smoke gate. Record/replay fixtures for read-only tools, or an explicit non-reproducible state, would close this gap. Two further asymmetries are deliberate. Operational failures (provider outage) do not waive the gate; they release the review claim so the review can be retried, and are recorded separately from candidate failures. And the held-out comparison tolerates environmental staleness: a pooled case that errors identically under both revisions neutralizes itself; only the candidate doing *worse* than its predecessor on material it never saw is a regression.

Finally, the keyword fallback. Our own measurements (§5) show it behaves as near-verbatim matching, rejecting every innocuous rewording. A design more consistent with the platform's fail-closed philosophy would have the automatic gate *abstain* when no sufficiently strong grounded evidence exists, routing such candidates to human or domain-specific verification instead of certifying them against keywords. We consider that the right next change to the system.

4.4 Promotion, provenance, rollback

Candidates move `Proposed` $\rightarrow$ `Evaluating` $\rightarrow$ `Promoted` | `Rejected`; both accept and reject first claim the candidate, bound to a request digest, so concurrent reviewers cannot both succeed and retries replay rather than re-execute. Acceptance requires `AcceptanceChecks` derived from what actually executed (including whether any held-out material existed) and activates the new revision inside

one transaction through the platform's shared release contract: validation report, release-candidate record with the gate report's hash as the plan hash, decision record with per-gate results, and a compare-and-set move of the tenant's *serving pointer*. There is no `published` status on skill artifacts; visibility is owned by the pointer, which makes activation atomic and rollback a pointer operation rather than a data migration. Every promotion appends `skill_created` or `skill_improved` to an append-only, bitemporal learning log with typed source provenance; rollback invalidates by setting `valid_to`, never by deletion.

After promotion, a deterministic monitor (no model calls) compares the skill's segment resolution rate in a 14-day window after activation against the pre-promotion baseline. It abstains below five samples; for improvements it files a rollback proposal when the post rate drops more than 0.2 below baseline, and for newly created skills (no baseline) when the post rate falls below an absolute floor of 0.3. This monitor should be read as an *operational heuristic*, not a statistically supported regression detector: five samples is very noisy for a rate metric, and naive before/after comparison is confounded by task mix, model and tool changes, and traffic shifts. A stronger version would condition on the same task family with the skill actually activated and attach interval estimates; recent measurements that skills materially regress previously solved tasks [36] motivate investing there. Rollback proposals are themselves reviewable candidates carrying the full evidence (rates, sample counts, revision and audit identifiers), keyed to the exact serving epoch so a re-activation cannot inherit stale evidence. Accepting one archives the regressed revision and tombstones its pointer, leaving the skill *unserved* rather than silently restoring a predecessor, which would bypass the gate. This is a deliberate availability-for-safety trade: after a rollback of an improvement, no revision serves until a replacement clears its own gate, rather than reverting to a predecessor whose gate evidence may no longer be valid for the current environment.

5 Case study: does the oracle measure what it claims?

Live promotion metrics require production traffic and are deferred to future work (§8). What we can establish now, rigorously, is whether the oracle has the discrimination properties the design claims: that it accepts faithful and semantically equivalent reruns, rejects fabricated or evidence-free results, refuses spurious grounding, and tolerates legitimate procedural variation. Because suite generation and assertion evaluation are deterministic, these properties are measurable exactly on the constructed test distribution; determinism removes run-to-run measurement noise, though not the construction uncertainty of generalizing beyond these cases (§8). The harness drives the production code paths: the real sanitization gate (`sanitize_segment_evidence` with the production PII classifier), the real generator (`generate_skill_test_suite_source`), and the real gate-time evaluators (`evaluate_assertions` over the built-in registry), fed synthetic task segments with planted, typed facts. It is seeded, makes no network or model calls, and is released with the paper. Segments embed facts of eight types in tool results, with final responses carrying a controlled number of those facts verbatim; the characterization experiment sweeps $t \in [2, 8]$ tool calls and $k \in [0, 5]$ carried facts.

Oracle selection and determinism (n=300). The generator chose the `GroundedFacts` oracle in exactly the 250 segments whose response carried at least one tool-corroborated fact and fell back to `Keywords` in exactly the 50 segments with none ($k{=}0$); repeated generation produced identical assertion sets and oracle choices in 300/300 cases. Grounded cases asserted a median of 4 facts (cap 5), dominated by numeric and currency spans (including incidental grounded spans beyond the planted facts), with UUIDs and reference tokens where planted.

Adversarial grounding, alphanumeric (n=200). With tool outputs corrupted so that every response fact appears only as a strict substring of a longer alphanumeric token (`412ms` inside `1412msx`), the word-boundary check refused to ground in 200/200 segments: zero false groundings on this adversarial family.

Adversarial grounding, punctuation extensions (n=300). The boundary rule does not protect against extensions that begin with a non-word character; we owe this measurement to adversarial review of an earlier draft. Across six families—currency (`$1,200` vs. tool output `$1,200.99`), reference tokens (`ABC-123` vs. `ABC-123-456`), paths (`/tmp/foo` vs. `/tmp/foo/bar`), URL prefixes, decimals (`12` vs. `12.5`), and hyphenated units (`412ms` vs. `412ms-related`)—the response fact falsely grounded against the punctuation-extended tool token in **300/300 cases**. The precision boundary is the

Table 1: Gate outcomes under response perturbations, grounded-facts oracle versus its own keyword fallback (200 segments each; deterministic, so rates are exact). Fabrication and omission rows are spec-conformance checks (string-level mutations of planted facts); the paraphrase row is the empirical contrast: keywords reject every fact-preserving paraphrase, behaving as brittle near-verbatim matching.

Candidate response	Desired verdict	Grounded facts	Keyword fallback
Exact reproduction	pass	**100% pass**	100% pass
Planted-fact-preserving paraphrase	pass	**92.5% pass**	0% pass
Facts fabricated (digits mutated)	fail	**100% fail**	100% fail
Facts omitted (fluent, evidence-free)	fail	**100% fail**	100% fail

word-character definition itself: safe against alphanumeric extension, systematically unsafe against punctuation extension. The fix is typed per-class canonicalization (parse and compare complete monetary tokens, normalized URLs, full path segments, and value-unit pairs, rather than applying one universal boundary regex) plus per-fact provenance recording the originating tool call and byte span, which would also address the source-conflation risk that pooled grounding corpora share [2].

Perturbation detection (n=200 segments $\times$ 4 variants $\times$ 2 oracles). Table 1 summarizes the central result. Two kinds of rows must be distinguished. The fabrication and omission rows (and the alphanumeric adversarial check above) are *spec-conformance* results: the perturbations are string-level mutations of planted facts, so a correct implementation of exact matching is expected to detect them at 100%, and the value of the measurement is confirming the deployed implementation meets its spec with no false groundings. The *empirical* finding is the selectivity contrast between the two oracles on paraphrases. Under the grounded-facts oracle, exact reruns always pass; responses whose facts were fabricated (every digit mutated, preserving format) or omitted (fluent text, no evidence) always fail the blocking fact assertion. Planted-fact-preserving paraphrases pass 92.5% of the time on this finite deterministic suite; inspection shows all 15 false rejections trace to a single cause. The oracle had additionally grounded an incidental enumeration digit ("Finding 3" matching a step index in tool output, since the trivial-number floor excludes only 0–2 and years), and the paraphrase dropped it. The keyword fallback detects fabrication identically but rejects *every* paraphrase: asserting a fixed set of the response's longer tokens makes it a near-verbatim match on arbitrary wording. The grounded oracle's advantage is not detection but *selectivity*: it pins the spans that carry the task's result and frees everything else to vary. This is the measured justification for preferring grounded facts wherever they exist, and the case against letting keywords participate in automatic certification at all (§4.3).

Action assertions (n=200). The distinct-tool-set assertion detected a dropped tool in 200/200 runs, while a maximally rerouted trajectory (reversed order with a duplicated call, *same distinct set*) passed 200/200. Scope matters here: what is tolerated is alternative *orderings and repetition* over the recorded tool set, not alternative routes. A candidate that reaches the same verified result through a different tool set, including a genuine improvement that eliminates an unnecessary call, fails the assertion by design. This is a deliberate conservatism (the dropped-tool row is that same behavior, viewed as detection); the natural relaxation is capability-level requirements derived from each fact's originating tool.

6 Design lessons

A trajectory-similarity gate was built, then deleted. An earlier revision gated candidates on longest-common-subsequence similarity between the candidate's tool trajectory and the recorded one. It punished exactly the wrong thing: a candidate that reached the same verified result by a shorter or reordered route failed, while the assertion said nothing about whether the *result* was real. The replacement demands the distinct tool set (was the capability used at all?) as blocking, and reports order as a diagnostic—one observation of one run, not a requirement. The case study's reroute result (§5) measures the replacement's tolerance directly.

Fail closed, everywhere. Suites declare a schema version, and older versions are rejected outright rather than partially parsed: a silently ignored expectation block would turn into a vacuous pass. Assertions name registry evaluators with versions; an unknown evaluator fails the case rather than

skipping it. An empty evaluator configuration still runs assertions, so a suite cannot dodge its claims. The same posture governs sanitization (refusal ends the pass with zero writes) and the monitor (abstains below minimum samples rather than guessing).

Keep quality signals out of the artifact. Use counts, success rates, and rollback history live in revisions, candidates, and the learning log, never in SKILL.md, whose unsupported metadata keys are rejected at parse. The artifact a reviewer reads is exactly the artifact the agent executes; runtime state cannot smuggle itself into a promoted document.

Determinism is a reviewability feature, with limits. Because suite generation and assertion evaluation are deterministic, a reviewer sees the exact facts the gate will demand, an auditor can regenerate the suite from the sanitized evidence and obtain the same assertions, and the platform can replay a promotion decision from its request digest. The gate *verdict* is not fully deterministic: it rests on stochastic eval-agent runs against a live provider. What determinism buys is that every nondeterministic contribution is confined to those runs, and everything that specifies, scores, and records them is reproducible. Within the gate itself, the model appears in two places (writing the skill document and executing the eval runs), and neither can alter what the gate demands. The wider pipeline does use models elsewhere, in routing embeddings and in a self-assessment component among the segment-assessment signals, which is why we scope the claim to the gate.

7 Threat model

A multi-tenant system that learns durable artifacts from its own traffic must consider adversaries, and PII sanitization is not poisoning protection. We enumerate the threats the design faces and its current posture on each. *Poisoned evidence*: an attacker can place reusable malicious instructions in user text, tool outputs, ticket bodies, or fetched web content, hoping distillation memorializes them into a served skill. The platform routes tool traffic through a security layer with prompt-injection controls and outbound-destination policy, and the distillation prompt instructs the model to learn workflow structure rather than copy content. But the learning path's primary defenses are the gate (injected instructions do not help a candidate reproduce grounded facts), human review of every promotion, and rollback; none of these is proof against a skill that is both malicious and task-effective. *Compromised or wrong tools*: the oracle trusts successful tool results as its grounding corpus, so a tool that returns attacker-controlled or incorrect data grounds incorrect facts; provenance-aware per-tool grounding (§5) would narrow but not remove this. *Sanitizer false negatives*: the type boundary prevents bypassing the sanitizer, not classifier misses (§4.1). *Recurrence flooding*: an attacker who can generate many similar sessions can manufacture a recurring task and its sibling suites; tenant isolation confines the blast radius to the attacker's own tenant, and per-tenant candidate caps bound the volume, but within a tenant this remains a review-burden attack. *Held-out leakage*: a proposer that observes held-out failures across repeated proposals gradually exposes the pool (§4.3); rotating holdout material across generations would mitigate this. *Privilege escalation via skill text*: a skill's prose could instruct the agent toward tools beyond its remit, but runtime tool policy is compiled at the agent and turn level and enforced at dispatch; it is not read from skill text, so prose alone cannot widen permissions. The frontmatter `allowed-tools` declaration does influence the agent's resolved tool set, and it enters that set only as part of the reviewed, gated artifact. We present this enumeration as a design map, not a security evaluation; adversarial testing of the learning path is future work.

8 Limitations

The case study characterizes the oracle's discrimination properties on synthetic segments constructed to be groundable (the planted facts match the extractor's span classes by design), so its detection rows verify the implementation rather than estimate real-world recall, and the paraphrase variants keep fact spans byte-identical, making 92.5% an upper bound on a favorable distribution. The oracle demands surface-form-exact reproduction: a rerun that reformats a fact (`412ms` vs. `0.41s`, `$1,200` vs. `1200 USD`) is rejected, and quantifying that cost on harder, model-generated paraphrases is needed before the selectivity number can be taken at face value. False rejections also compound across a gate's cases: at the observed 7.5% per-case rate, a candidate facing four independent cases would be falsely rejected roughly a quarter of the time, a reviewer-burden cost the design pays for conservatism. The study also does not compare against an LLM-judge baseline on the same perturbation grid, which would quantify what the deterministic oracle gives up; nor does it measure

end-to-end improvement of a deployed fleet. The decisive production questions (what fraction of promoted skills raise resolution rates, how often the monitor fires, what the reviewer burden is) require live tenant traffic and are the subject of ongoing measurement; this paper documents the mechanism and its verifiable properties, and we intend a follow-up with longitudinal deployment data. The oracle's precision has two measured soft edges: incidental grounded spans (enumeration digits, step indices) can over-constrain paraphrased reruns, and punctuation-extended tool tokens falsely ground response facts at 100% in the tested families (§5); ranked typed fact selection with per-class canonicalization and per-fact provenance addresses both and is the highest-priority implementation refinement. The false-rejection compounding estimate above also assumes case independence, which correlated incidental spans violate. The oracle also inherits the limits of its corpus: tasks whose success leaves no verbatim trace in tool outputs (pure advice, summarization) fall back to keywords, and a response could in principle echo tool output without correctly completing the task; the tool-set assertion and human review bound but do not eliminate this. Held-out siblings measure generalization across sessions of the *same* recurring task, not transfer to novel tasks. Finally, single-case suites per session keep gate cost inside a $0.50 budget but bound per-candidate statistical power; the design compensates with accumulation over time rather than breadth per gate.

9 Conclusion

Self-improvement in production is a release-engineering problem before it is a learning problem. With held-out gating of self-modification now an active research direction [22, 43, 46], the contribution we defend sits one layer beneath the gate: deterministic regression assertions constructed from tool-corroborated facts in unlabeled traces, wrapped in a lifecycle that treats each learned skill as a governed artifact. A skill is admitted only from sanitized, high-confidence experience; cleared by paired non-regression against the serving revision, on material including held-out sibling sessions; promoted atomically behind a serving pointer by a human decision with full provenance; and watched by a monitor that can propose its removal. Within its string-level perturbation model, the case study shows the oracle detects every fabricated or evidence-free result and accepts nearly all planted-fact-preserving paraphrases, a selectivity the keyword baseline lacks entirely; the punctuation-extension study locates the matcher's real precision boundary and fixes its direction of improvement. Concrete next steps: typed canonicalization and per-fact provenance in the matcher; abstention in place of the keyword fallback; longitudinal measurement of promotion outcomes on live tenant traffic; and extending sibling pooling across tenants under differential-privacy constraints, so that recurring tasks common to many organizations can be learned once, gated everywhere.

References

[1] Agent Skills. Agent skills specification. `https://agentskills.io/specification`, 2026. Accessed 2026-08-07.

[2] Ander Alvarez, Santhiya Rajan, Samuel Mugel, and Román Orús. ProvenanceGuard: Source-aware factuality verification for MCP-based LLM agents, 2026.

[3] Saleema Amershi, Andrew Begel, Christian Bird, Robert DeLine, Harald Gall, Ece Kamar, Nachiappan Nagappan, Besmira Nushi, and Thomas Zimmermann. Software engineering for machine learning: A case study. In *2019 IEEE/ACM 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, pages 291–300. IEEE, 2019. doi: 10.1109/ICSE-SEIP.2019.00042.

[4] Anthropic. Introducing claude skills. `https://www.anthropic.com/news/skills`, 2025. Accessed 2026-08-07.

[5] Avrim Blum and Moritz Hardt. The ladder: A reliable leaderboard for machine learning competitions, 2015. Published at the International Conference on Machine Learning.

[6] Bernd Bohnet, Vinh Q. Tran, Pat Verga, Roee Aharoni, Daniel Andor, Livio Baldini Soares, Massimiliano Ciaramita, Jacob Eisenstein, Kuzman Ganchev, Jonathan Herzig, Kai Hui, Tom Kwiatkowski, Ji Ma, Jianmo Ni, Lierni Sestorain Saralegui, Tal Schuster, William W. Cohen, Michael Collins, Dipanjan Das, Donald Metzler, Slav Petrov, and Kellie Webster. Attributed question answering: Evaluation and modeling for attributed large language models, 2022.

[7] Eric Breck, Shanqing Cai, Eric Nielsen, Michael Salib, and D. Sculley. The ml test score: A rubric for ml production readiness and technical debt reduction. In *2017 IEEE International*

Conference on Big Data (Big Data), pages 1123–1132. IEEE, 2017. doi: 10.1109/BIGDATA.2017.8258038.

[8] Deng Cai, Elman Mansimov, Yi-An Lai, Yixuan Su, Lei Shu, and Yi Zhang. Measuring and reducing model update regression in structured prediction for NLP, 2022. Published at Neural Information Processing Systems.

[9] Bei Chen, Fengji Zhang, Anh Nguyen, Daoguang Zan, Zeqi Lin, Jian-Guang Lou, and Weizhu Chen. Codet: Code generation with generated tests, 2022. Published at the International Conference on Learning Representations.

[10] Lingjiao Chen, Matei Zaharia, and James Zou. How is ChatGPT's behavior changing over time?, 2023.

[11] Minghao Chen, Yihang Li, Yanting Yang, Shiyu Yu, Binbin Lin, and Xiaofei He. Automanual: Constructing instruction manuals by LLM agents via interactive environmental learning, 2024. Published at Neural Information Processing Systems.

[12] Haowen Gao, Haoran Chen, Can Wang, Shasha Guo, Liang Pang, Zhaoyang Liu, Huawei Shen, and Xueqi Cheng. SkillAudit: Ground-truth-free skill evolution via paired trajectory auditing, 2026.

[13] Huan-ang Gao, Jiayi Geng, Wenyue Hua, Mengkang Hu, Xinzhe Juan, Hongzhang Liu, Shilong Liu, Jiahao Qiu, Xuan Qi, Yiran Wu, Hongru Wang, Han Xiao, Yuhang Zhou, Shaokun Zhang, Jiayi Zhang, Jinyu Xiang, Yixiong Fang, Qiwen Zhao, Dongrui Liu, Qihan Ren, Cheng Qian, Zhenhailong Wang, Minda Hu, Huazheng Wang, Qingyun Wu, Heng Ji, and Mengdi Wang. A survey of self-evolving agents: What, when, how, and where to evolve on the path to artificial super intelligence, 2025.

[14] Luyu Gao, Zhuyun Dai, Panupong Pasupat, Anthony Chen, Arun Tejasvi Chaganty, Yicheng Fan, Vincent Zhao, Ni Lao, Hongrae Lee, Da-Cheng Juan, and Kelvin Guu. Rarr: Researching and revising what language models say, using language models. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 16477–16508. Association for Computational Linguistics, 2023. doi: 10.18653/v1/2023.acl-long.910.

[15] Tianyu Gao, Howard Yen, Jiatong Yu, and Danqi Chen. Enabling large language models to generate text with citations, 2023. Published at the Conference on Empirical Methods in Natural Language Processing.

[16] Sayash Kapoor, Benedikt Stroebl, Zachary S. Siegel, Nitya Nadgir, and Arvind Narayanan. AI agents that matter, 2024. Published in Transactions on Machine Learning Research.

[17] Changmao Li and Jeffrey Flanigan. Task contamination: Language models may not be few-shot anymore, 2024. AAAI 2024; arXiv preprint 2023.

[18] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. Self-refine: Iterative refinement with self-feedback, 2023. Published at Neural Information Processing Systems.

[19] Bodhisattwa Prasad Majumder, Bhavana Dalvi Mishra, Peter Jansen, Oyvind Tafjord, Niket Tandon, Li Zhang, Chris Callison-Burch, and Peter Clark. Clin: A continually learning language agent for rapid task adaptation and generalization, 2023.

[20] Potsawee Manakul, Adian Liusie, and Mark J. F. Gales. SelfCheckGPT: Zero-resource black-box hallucination detection for generative large language models, 2023. Published at the Conference on Empirical Methods in Natural Language Processing.

[21] Sewon Min, Kalpesh Krishna, Xinxi Lyu, Mike Lewis, Wen-tau Yih, Pang Wei Koh, Mohit Iyyer, Luke Zettlemoyer, and Hannaneh Hajishirzi. FActScore: Fine-grained atomic evaluation of factual precision in long form text generation, 2023. EMNLP 2023.

[22] Johannes Moll, Jean-Philippe Corbeil, Jiazhen Pan, Martin Hadamitzky, Daniel Rueckert, Lisa Adams, and Keno Bressem. GRASP: Gated regression-aware skill proposer for self-improving LLM agents, 2026.

[23] Carlos Pacheco, Shuvendu K. Lahiri, Michael D. Ernst, and Thomas Ball. Feedback-directed random test generation. In *29th International Conference on Software Engineering (ICSE)*, pages 75–84. IEEE, 2007. doi: 10.1109/ICSE.2007.37.

[24] Andrei Paleyes, Raoul-Gabriel Urma, and Neil D. Lawrence. Challenges in deploying machine learning: A survey of case studies. *ACM Computing Surveys*, 55(6):1–29, 2022. doi: 10.1145/3533378.

[25] Alexander Pan, Kush Bhatia, and Jacob Steinhardt. The effects of reward misspecification: Mapping and mitigating misaligned models, 2022. Published at the International Conference on Learning Representations.

[26] Siran Peng, Cuiyu Yang, Tianyu Fu, Tianshuo Zhang, Haoyuan Zhang, Weisong Zhao, Anyang Su, Minghui Wu, Huiying Li, Xiangyu Zhu, Chenxu Zhao, and Zhen Lei. Self-supervised skill optimization, 2026.

[27] Hannah Rashkin, Vitaly Nikolaev, Matthew Lamm, Lora Aroyo, Michael Collins, Dipanjan Das, Slav Petrov, Gaurav Singh Tomar, Iulia Turc, and David Reitter. Measuring attribution in natural language generation models. *Computational Linguistics*, 49(4):777–840, 2023. doi: 10.1162/coli_a_00486.

[28] Benjamin Recht, Rebecca Roelofs, Ludwig Schmidt, and Vaishaal Shankar. Do ImageNet classifiers generalize to ImageNet?, 2019. Published at the International Conference on Machine Learning.

[29] Marco Tulio Ribeiro, Tongshuang Wu, Carlos Guestrin, and Sameer Singh. Beyond accuracy: Behavioral testing of NLP models with CheckList. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 4902–4912. Association for Computational Linguistics, 2020. doi: 10.18653/v1/2020.acl-main.442.

[30] Oscar Sainz, Jon Ander Campos, Iker García-Ferrero, Julen Etxaniz, Oier Lopez de Lacalle, and Eneko Agirre. NLP evaluation in trouble: On the need to measure LLM data contamination for each benchmark, 2023. Published at the Conference on Empirical Methods in Natural Language Processing.

[31] D. Sculley, Gary Holt, Daniel Golovin, Eugene Davydov, Todd Phillips, Dietmar Ebner, Vinay Chaudhary, Michael Young, Jean-François Crespo, and Dan Dennison. Hidden technical debt in machine learning systems. In *Advances in Neural Information Processing Systems*, volume 28, 2015.

[32] Shreya Shankar, Rolando Garcia, Joseph M. Hellerstein, and Aditya G. Parameswaran. Operationalizing machine learning: An interview study, 2022.

[33] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning, 2023. Published at Neural Information Processing Systems.

[34] Ilia Shumailov, Zakhar Shumaylov, Yiren Zhao, Yarin Gal, Nicolas Papernot, and Ross Anderson. The curse of recursion: Training on generated data makes models forget, 2023.

[35] Weihao Tan, Wentao Zhang, Xinrun Xu, Haochong Xia, Ziluo Ding, Boyu Li, Bohan Zhou, Junpeng Yue, Jiechuan Jiang, Yewen Li, Ruyi An, Molei Qin, Chuqiao Zong, Longtao Zheng, Yujie Wu, Xiaoqiang Chai, Yifei Bi, Tianbao Xie, Pengjie Gu, Xiyun Li, Ceyao Zhang, Long Tian, Chaojie Wang, Xinrun Wang, Börje F. Karlsson, Bo An, Shuicheng Yan, and Zongqing Lu. Cradle: Empowering foundation agents towards general computer control, 2024. Published at the International Conference on Machine Learning.

[36] Darshan Tank and Baran Nama. The regression tax: Decomposing why skills help and hurt LLM agents, 2026.

[37] Zhengwei Tao, Ting-En Lin, Xiancai Chen, Hangyu Li, Yuchuan Wu, Yongbin Li, Zhi Jin, Fei Huang, Dacheng Tao, and Jingren Zhou. A survey on self-evolution of large language models, 2024.

[38] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models, 2023. Published in Transactions on Machine Learning Research.

[39] Peiyi Wang, Lei Li, Liang Chen, Zefan Cai, Dawei Zhu, Binghuai Lin, Yunbo Cao, Qi Liu, Tianyu Liu, and Zhifang Sui. Large language models are not fair evaluators, 2023. Published at the Annual Meeting of the Association for Computational Linguistics.

[40] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models, 2022. Published at the International Conference on Learning Representations.

[41] Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. Agent workflow memory, 2025. ICML 2025; arXiv preprint 2024.

[42] Sijie Yan, Yuanjun Xiong, Kaustav Kundu, Shuo Yang, Siqi Deng, Meng Wang, Wei Xia, and Stefano Soatto. Positive-congruent training: Towards regression-free model updates. In *2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 14294–14303. IEEE, 2021. doi: 10.1109/CVPR46437.2021.01407.

[43] Yifan Yang, Ziyang Gong, Weiquan Huang, Qihao Yang, Ziwei Zhou, Zisu Huang, Yan Li, Xuemei Gao, Qi Dai, Bei Liu, Kai Qiu, Yuqing Yang, Dongdong Chen, Xue Yang, and Chong Luo. SkillOpt: Executive strategy for self-evolving agent skills, 2026.

[44] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ-bench: A benchmark for tool-agent-user interaction in real-world domains, 2024.

[45] Eric Zelikman, Yuhuai Wu, Jesse Mu, and Noah D. Goodman. Star: Bootstrapping reasoning with reasoning, 2022.

[46] Hangfan Zhang, Shao Zhang, Kangcong Li, Chen Zhang, Yang Chen, Yiqun Zhang, Lei Bai, and Shuyue Hu. Self-harness: Harnesses that improve themselves, 2026.

[47] Jenny Zhang, Shengran Hu, Cong Lu, Robert Lange, and Jeff Clune. Darwin Gödel machine: Open-ended evolution of self-improving agents, 2025.

[48] Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. Expel: LLM agents are experiential learners, 2024. AAAI 2024; arXiv preprint 2023.

[49] Boyuan Zheng, Michael Y. Fatemi, Xiaolong Jin, Zora Zhiruo Wang, Apurva Gandhi, Yueqi Song, Yu Gu, Jayanth Srinivasa, Gaowen Liu, Graham Neubig, and Yu Su. Skillweaver: Web agents can self-improve by discovering and honing skills, 2025.

[50] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-judge with MT-Bench and chatbot arena, 2023. Published at Neural Information Processing Systems.

A Gate and oracle constants

Table 2 lists the default configuration governing the mechanism, as shipped. All are tenant-configurable except structural invariants (schema versioning, the held-out exclusion of `Generated` suites, and the sanitization gate, which cannot be disabled).

B Reproducing the case study

The harness is a single-file Rust crate that path-depends on the platform's crates and ships the workspace lockfile; `cargo run` regenerates every number in §5 deterministically (seeds 7, 11, 23, 31, 47; no network, no model calls, $\sim$1 minute on a laptop). Segments are synthesized as typed event logs (user message, tool call/result pairs, final response) and pass through the production sanitization, generation, and evaluation entry points; perturbations mutate only the candidate response text or action ledger presented to the evaluators. Paraphrase variants are template-generated: surrounding prose is rewritten while fact spans are preserved byte-identically (§8 discusses why this is the favorable case). Per-experiment counts: characterization 300 segments, alphanumeric adversarial grounding 200, punctuation-extension adversarial grounding 300, perturbation study 200 grounded + 200 keyword-twin segments, action assertions 200 (total 1,400); gate evaluations $200 \times 4 \times 2 + 200 \times 2 = 2{,}000$ (the adversarial studies exercise suite generation only). Code and the emitted `results.json` accompany the paper.

Table 2: Default constants of the learning loop.

Component	Constant	Default
Learnability	`Resolved` confidence floor	0.70
	`Partial` floor (needs attribution + verification)	0.85
	minimum tool calls (single-session / recurrence)	8 / 3
Routing	improve-vs-create cosine similarity	0.80
	proposal dedupe similarity	0.85
Recurrence	occurrences / lookback / cooldown	3 / 30 d / 30 d
	cluster similarity	0.85
Oracle	max facts per case / max fact length	5 / 80 bytes
	trivial numbers excluded	0–2, years 1900–2099
Gate	budget per gate / suite timeout floor	\$0.50 / 90 s
	held-out sibling cap	3
	pass criterion	fewer failures, else score $\geq$ baseline
Monitor	min samples / lookback	5 / 14 d
	regression delta (improved) / floor (created)	0.2 / 0.3